

CS151 Intro to Data Structures

LinkedLists

Announcements

- HW01 due Friday September 18th
- Lab2 due Today at midnight
- Quiz 1 in lab on Thursday
- Professor office hours today 3-5pm
 - Park 259

Outline

- Review - ExpandableArray
 - instance of
- **LinkedLists**

Warm up

Write a class `Account` with:

1. A nested exception hierarchy:
 - `AccountException` extending `Exception`
 - `InsufficientFunds` and `AccountFrozen`, both extending `AccountException`
2. A `withdraw(int amount)` method that throws `AccountFrozen` if the account is frozen, `InsufficientFunds` if the amount exceeds the balance, and otherwise subtracts.

Warm up

Write a generic class `Box<T>` with a fixed-size array member variable:

- Constructor takes an `int capacity` for the size of the array
- `add(T item)` appends to the next open slot.
- `get(int i)` returns the element at index `i`.

Runtime complexity of each operation?

Arrays Review

- Java arrays: Sequential, contiguous, memory layout
 - gives us $O(1)$ access (if we know the index)
- fixed size
- `insert(E elem, int idx)`
- `remove(int idx)`
- `set(E elem, int idx)`

instanceof

- An operator that tests to see if an object is an instance of a specified type
- Every subclass object is an instance of its super class – not true the other way

```
class A {} class B extends A {} class C extends B {}  
A[] as = {new A(), new B(), new C()};  
for (int i=0; i<as.length; i++) {  
    System.out.print((as[i] instanceof A)+ " ");  
    System.out.print((as[i] instanceof B)+ " ");  
    System.out.println(as[i] instanceof C);  
}
```

Generics Review

- A way to write classes or methods that can operate on a variety of data types without being locked into specific types at the time of definition
- Write definitions with type parameters

```
public <T> void print(T x) {  
    System.out.println(x);  
}
```

Generics Review

Generic classes:

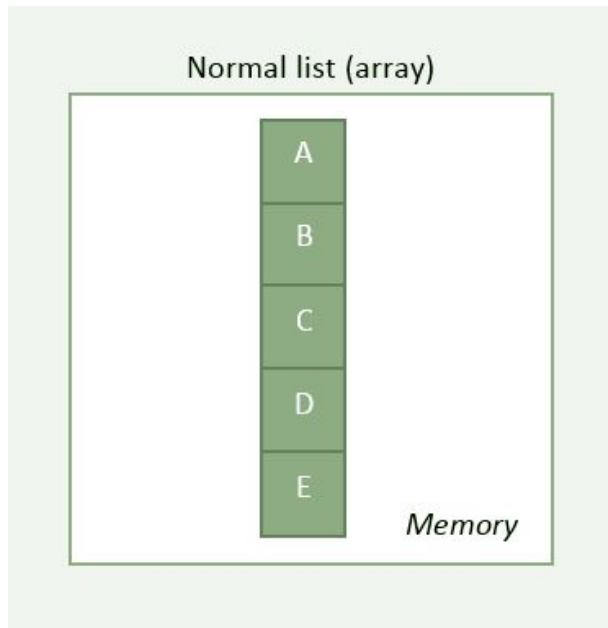
```
public class GenericExpandableArray <T> {
```

T can be used as a type within the class

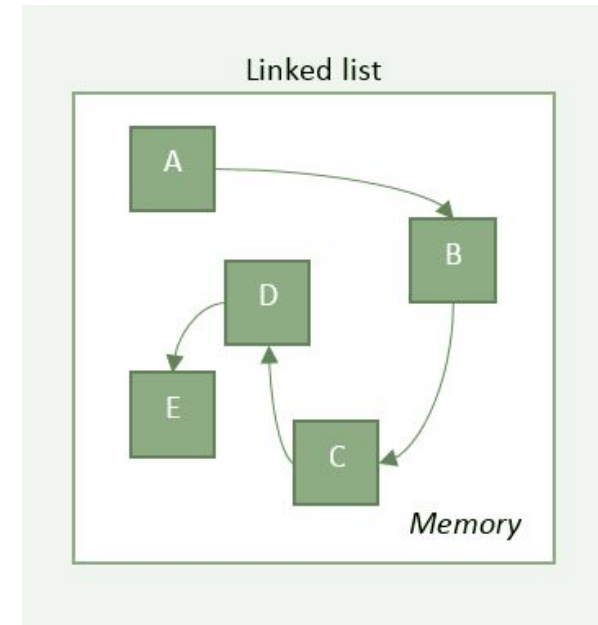
Linked List

List versus Array - memory

An array is a single consecutive piece of memory



A list can be made of many disjoint pieces

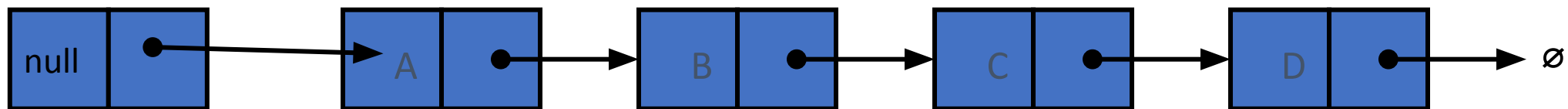


Linked List

- A linked list is a lists of objects (**nodes**)
- The **nodes** form a linear sequence
- Linked lists are typically unbounded, that is, they can grow infinitely.

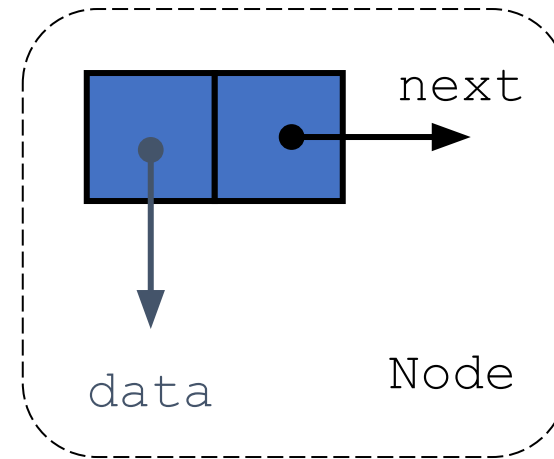
node: basic unit that contains data and one or more references or *links* to other nodes.

head



A node

```
public class Node<T> {  
    private T data;  
    private Node next;  
}
```

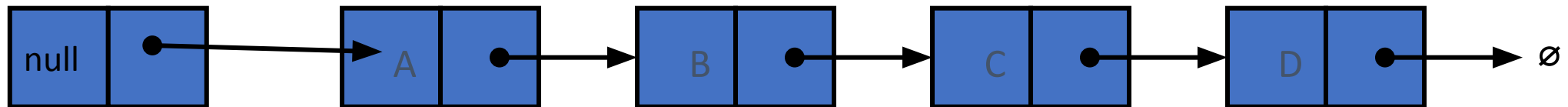


Linked List

How might we loop over all of the elements of a linked list?

```
public class Node<T> {  
    private T data;  
    private Node next;  
}
```

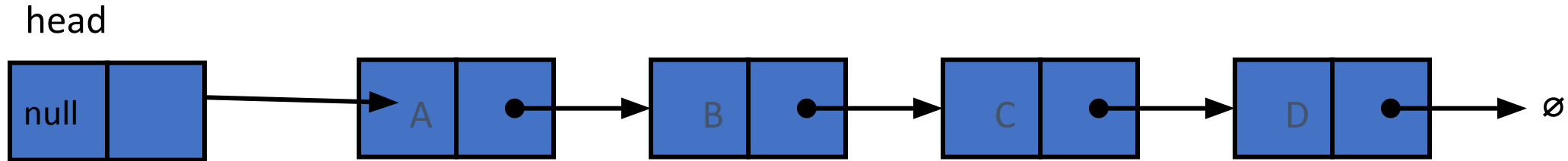
head



Linked List Operations

- Access
- Insertion
- Removal

Access Operation



- Check if the head node is what you are looking for
- Iterate through nodes:
 - Stop when found
 - Otherwise return null

Access Operation

Let's code it

- Runtime analysis
 - $O(n)$

Insert Operation

Let's code it

- Runtime analysis
 - Insert at head?
 - $O(1)$
 - Insert at tail?
 - $O(n)$
 - Insert at arbitrary location? (middle of list)
 - $O(n)$

Insert Operation

What if we keep a pointer to the tail?

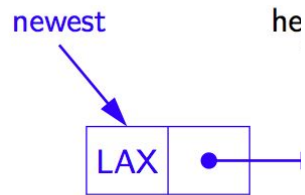
```
private Node tail;
```

How does this change our insertTail method?

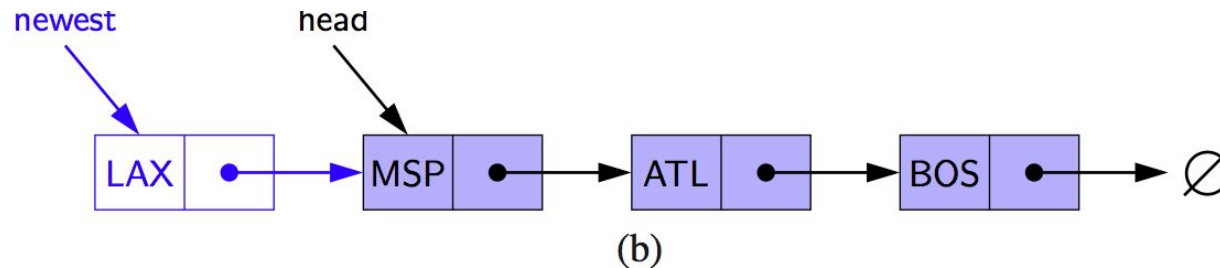
- Runtime analysis
 $O(1)$

Inserting at the Head

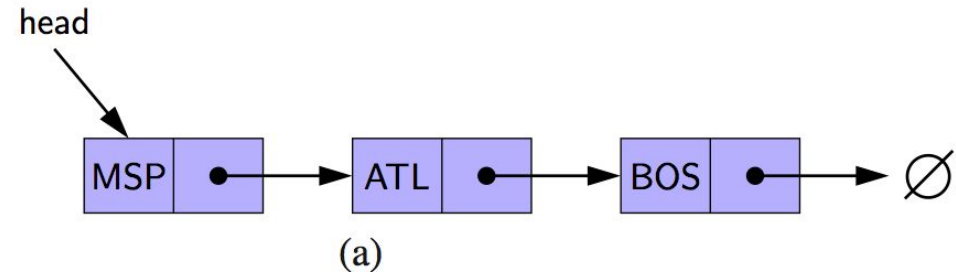
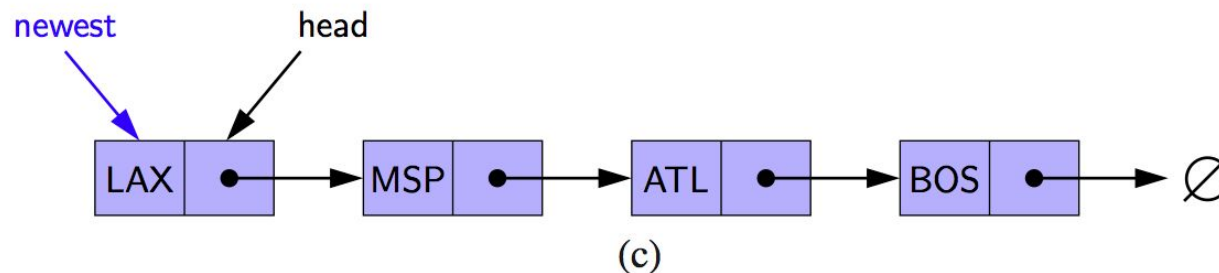
1. create a new node



1. have new node point to old head



1. update head to point to new node

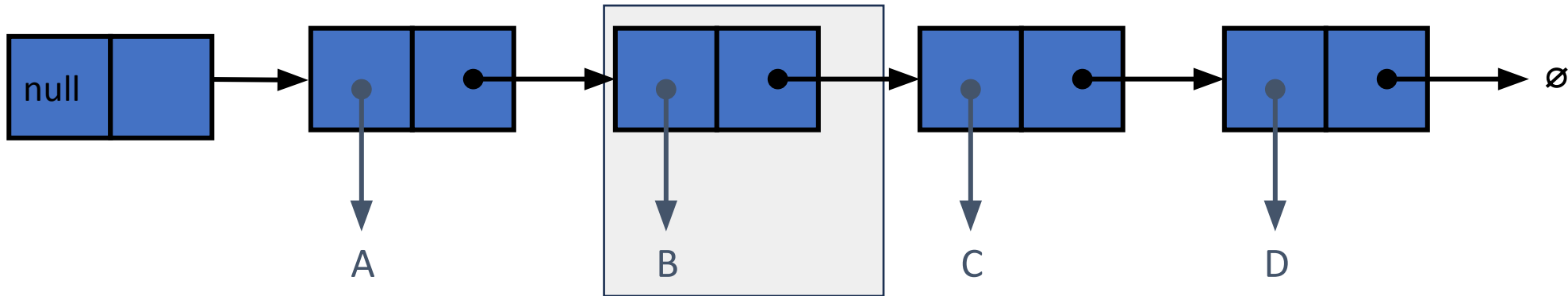


Remove Operation

- Let's write it on the board quickly

Remove Operation `remove("B")`

head



head



Properties in LinkedList

What do we need to keep track of?

- Head
- Number of elements (optional)

Quiz

Rank from most efficient to least efficient:

- LinkedList find
- ExpandableArray find
- LinkedList insert at beginning
- ExpandableArray insert at beginning

Summary

- Linked Lists are data structures with disjoint memory
- not fixed size! Grows as elements are added
- $O(n)$ access
- Insert at beginning is fast $O(1)$
- General insert is slow
 - in the worst case $O(n)$
- Removal is also slow $O(n)$